Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–5:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation (`simp`, `auto`, `nitpick`).
  - **Week 4:** Algebraic datatypes, termination measures, and tree induction.
  - **Week 5:** AI-assisted proving (Sledgehammer, RL loops, coding agents via MCP, DSP architecture).
- **Week 6 Milestone:** Verifying functional algorithm correctness and step bounds.
- Based on Tobias Nipkow's *Functional Data Structures* slides [Nipkow, 2020].

©Mohammad Abdulaziz

Outline

1. Correctness of sorting algorithms
   - Specification: `sorted` and multisets
   - Merge sort and computation induction
2. Running time
   - Counting function calls

©Mohammad Abdulaziz

Correctness of Sorting Algorithms

```
sorted ::  ('a::linorder) list ⇒ bool
```

©Mohammad Abdulaziz

Correctness of Sorting Algorithms

```
sorted ::  ('a::linorder) list ⇒ bool

sorted [] = True
sorted (x # ys) =
  ((∀y ∈ set ys.  x ≤ y) ∧ sorted ys)
```

©Mohammad Abdulaziz

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

©Mohammad Abdulaziz

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

Is that it?

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

Is that it? How about

```
set (sort xs) = set xs
```

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

Is that it? How about

```
set (sort xs) = set xs
```

Better: every `x` occurs as often in `sort xs` as in `xs`.

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

Is that it? How about

```
set (sort xs) = set xs
```

Better: every `x` occurs as often in `sort xs` as in `xs`.

More succinctly:

```
mset (sort xs) = mset xs
```

where `mset :: 'a list ⇒ 'a multiset`

What are multisets?

©Mohammad Abdulaziz

What are multisets?

Sets with (possibly) repeated elements

©Mohammad Abdulaziz

What are multisets?

Sets with (possibly) repeated elements

Some operations:

```
      {#}  ::  'a multiset
add_mset   ::  'a ⇒ 'a multiset ⇒ 'a multiset
        +  ::  'a multiset ⇒ 'a multiset ⇒ 'a multiset
```

©Mohammad Abdulaziz

What are multisets?

Sets with (possibly) repeated elements

Some operations:

```
      {#}  ::  'a multiset
add_mset   ::  'a ⇒ 'a multiset ⇒ 'a multiset
        +  ::  'a multiset ⇒ 'a multiset ⇒ 'a multiset
     mset  ::  'a list ⇒ 'a multiset
set_mset   ::  'a multiset ⇒ 'a set
```

©Mohammad Abdulaziz

What are multisets?

Sets with (possibly) repeated elements

Some operations:

```
      {#}  ::  'a multiset
add_mset   ::  'a ⇒ 'a multiset ⇒ 'a multiset
        +  ::  'a multiset ⇒ 'a multiset ⇒ 'a multiset
     mset  ::  'a list ⇒ 'a multiset
set_mset   ::  'a multiset ⇒ 'a set
```

Import `HOL-Library.Multiset`

©Mohammad Abdulaziz